FPGA VÀ VHDL

KIỂM TRA CUỐI KỲ

Thành viên : Nguyễn Minh Tài

Nguyễn Minh Toàn

Nguyễn Hữu Vượng

Giảng viên : TS. Nguyễn Khánh Quang

Lớp : 22PFIEV2

1. GIỚI THIỆU
2. THIẾT KẾ CPU
3. PHASE-LAG-CONTROL
4. VI CHƯƠNG TRÌNH
5. KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

FINAL-TERM PROJECT 2026

1. Design a simple processor (CPU)
2. Use this CPU to design the Phase Lag controller by software.
3. simulate it using ModelSim and SimuLink environment.

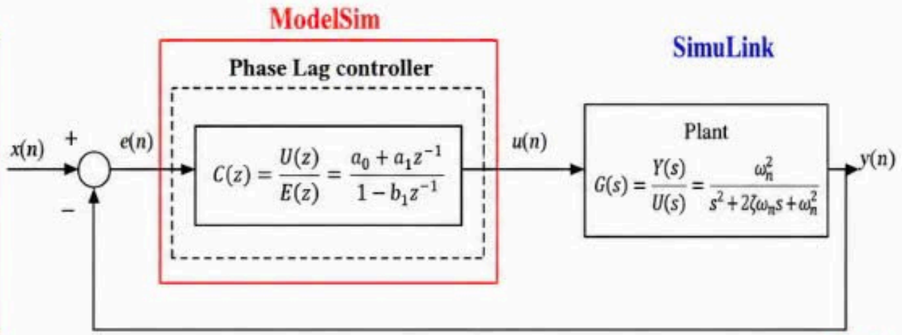


Fig.1

Plant: $G(s) = \dfrac{Y(s)}{U(s)} = \dfrac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$

with $\omega_n = 10, \quad \zeta = 0.5$

Phase Lag controller: $C(s) = \dfrac{U(s)}{E(s)} = K_c \dfrac{\tau_z s+1}{\tau_p s+1}$

with $K_c = 10, \quad \tau_z = 0.2s, \quad \tau_p = 2.0s$

Discreteform: $\dfrac{U(z)}{E(z)} = \dfrac{a_0 + a_1 z^{-1}}{1 - b_1 z^{-1}}$

Presentation date:
Jun .8, 2026

Differenceequation: $u(n) = b_1 u(n-1) + a_0 e(n) + a_1 e(n-1)$

error: $e(n) = x(n) - y(n)$

Input: x(n) is step command with 0.5 magnitude

Command: x(n) = 0.5 or 16384 for 24-bit-Q14

Plant: $G(s) = \dfrac{100}{s^2 + 10s + 100}$

Discrete plant: $y(n) = A_1 y(n-1) + A_2 y(n-2) + B_0 u(n)$

$A_1 = 1.8918918919, \quad A_2 = -0.9009099009, \quad B_0 = 0.0090090090$

($A_1 = 61994, \quad A_2 = -29521, \quad B_0 = 295$ for 24 - bit-Q14)

Phase Lag controller: $u(n) = b_1 u(n-1) + a_0 e(n) + a_1 e(n-1)$

$a_0 = 1.0224438903, \quad a_1 = -0.9725685786, \quad b_1 = 0.9950124688$

($a_0 = 33503, \quad a_1 = -31869, \quad b_1 = 32605$ for 24-bit-Q14)

PFIEV

GIỚI THIỆU

MỤC TIÊU

Thiết kế một hệ thống điều khiển vòng kín trên FPGA bằng VHDL.

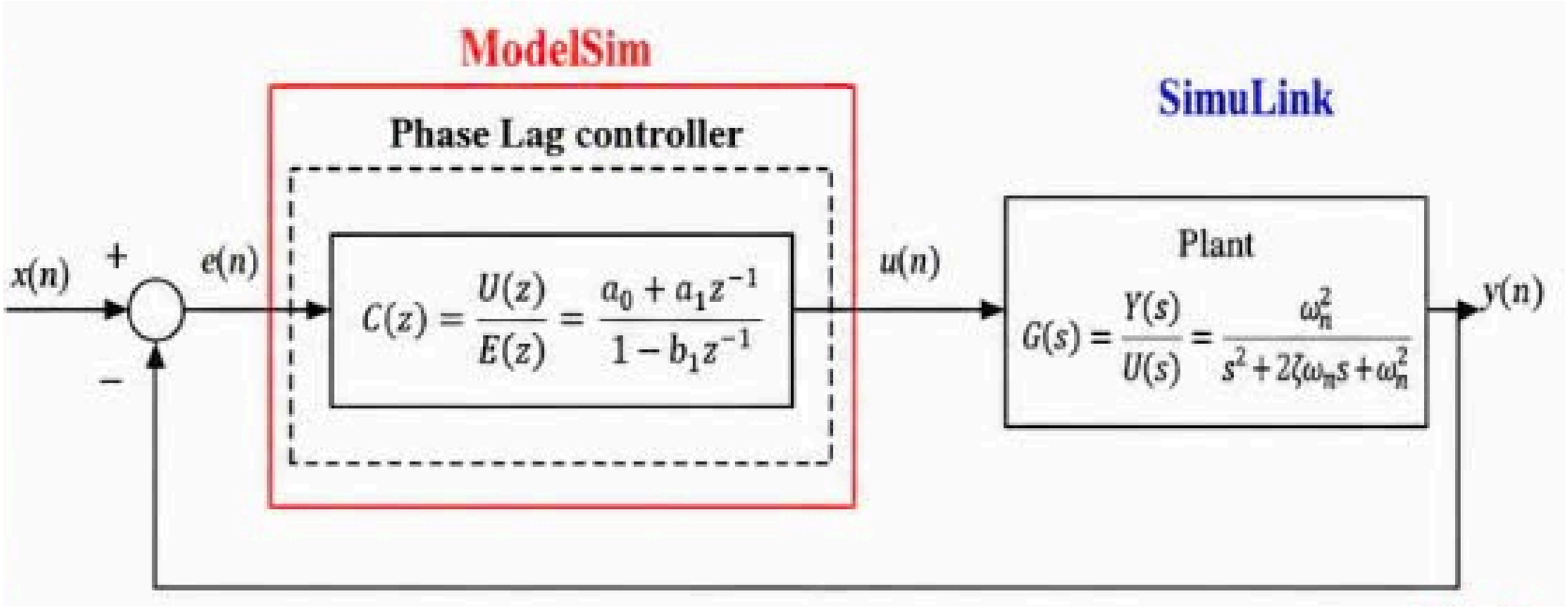
Đối tượng điều khiển: $G(s) = \dfrac{100}{s^2 + 10s + 100}$

Bộ điều khiển sử dụng: $C(s) = \dfrac{100(s+1)}{100s + 1}$

Đặc điểm hệ thống:

- Controller là Phase Lag Controller.
- Plant là hệ bậc hai.
- Dữ liệu dùng fixed-point signed 24-bit.
- Kiến trúc controller được xây dựng như một CPU nhỏ.
- CPU gồm:
  - microctrl.vhd: Control Subsystem.
  - microdata.vhd: Data Subsystem.
  - micro.vhd: ghép hai khối trên.

SƠ ĐỒ VÒNG KÍN



GIỚI THIỆU

PHASE LAG CONTROLLER DESIGN

1. PLANT

Hàm truyền: $G(s) = \frac{100}{s^2 + 10s + 100}$

Đặc tính: ωn = 10 rad/s, ζ = 0.5, G(0) = 1

Sai số xác lập (Uncompensated) khi chưa có controller :

$$e_{ss} = \frac{1}{1 + G(0)} = \frac{1}{2} = 50\%$$

2. Phase lag controller

=> Cần thiết kế bộ điều khiển phase Lag để tăng gain tần số thấp và giảm sai số xác lập.

Thiết kế bộ điều khiển: $C(s) = \frac{100(s+1)}{100s+1}$

Ý nghĩa:

C(0) = 100 → giảm sai số xác lập

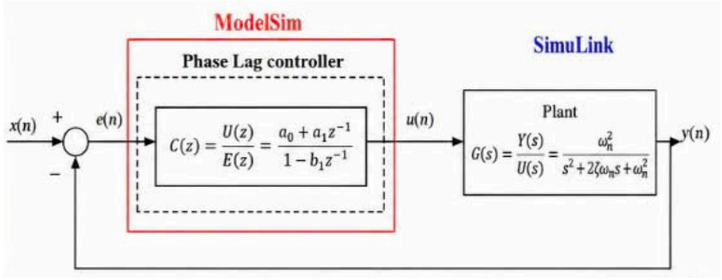
C(∞) = 1 → không khuếch đại nhiễu cao tần quá mạnh

Sau rời rạc hóa bằng Tustin, Ts = 0.01s:

u(k) = 0.999900u(k-1) + 1.004950e(k) - 0.994950e(k-1)

Trong đó:

e(k) = r(k) - y(k)



3. FIXED-POINT IMPLEMENTATION

Dùng fixed-point thay cho floating-point vì:

- Dễ hiện thực trên FPGA
- Tiết kiệm tài nguyên
- Chỉ cần nhân, cộng và dịch bit

Định dạng sử dụng:

- Signed 24-bit Q14
- Thực thi tương đương shift >> 14
- Scale hiệu dụng: 2^{14} = 16384

Quy đổi:

1.0 → 16384, 0.5 → 8192, -1.0 → -16384

Controller fixed-point:

B1 = 16382

A0 = 16465

A1 = -16301

u(k) = [16382u(k-1) + 16465e(k) - 16301e(k-1)] >> 1

PFIEV

KIẾN TRÚC HỆ THỐNG

```
phase_lag_project
│
├── phase_lag_pkg.vhd
├── phase_lag_top.vhd
├── micro.vhd
├── microctrl.vhd
├── microdata.vhd
├── plant.vhd
├── phase_lag_controller_cosim.vhd
├── phase_lag_tb.vhd
└── phase_lag_reference_model.m
```

```
phase_lag_top.vhd
│
├── micro.vhd
│   ├── microctrl.vhd
│   └── microdata.vhd
│
└── plant.vhd
```

THIẾT KẾ CPU

PLANT

Plant rời rạc:

$$y(k) = [P_1 y(k-1) + P_2 y(k-2) + B_0 u(k) + B_1 u(k-1) + B_2 u(k-2)] >> 14$$

```
ENTITY plant IS
    PORT (
        rst, sample_en : IN  STD_LOGIC;
        u_now          : IN  SIGNED(DATA_WIDTH - 1 DOWNTO 0);
        clk            : IN  STD_LOGIC;
        y_out          : OUT SIGNED(DATA_WIDTH - 1 DOWNTO 0);
        plant_done     : OUT STD_LOGIC;
        y_old1, y_old2 : OUT SIGNED(DATA_WIDTH - 1 DOWNTO 0);
        u_old1, u_old2 : OUT SIGNED(DATA_WIDTH - 1 DOWNTO 0)
    );
END plant;

ARCHITECTURE behavioral OF plant IS

    FUNCTION sat24(x : SIGNED(ACC_WIDTH - 1 DOWNTO 0)) RETURN SIGNED IS
        CONSTANT MAX24 : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := CONV_SIGNED(8388607, DATA_WIDTH);
        CONSTANT MIN24 : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := CONV_SIGNED(-8388608, DATA_WIDTH);
    BEGIN
        IF x > CONV_SIGNED(CONV_INTEGER(MAX24), ACC_WIDTH) THEN
            RETURN MAX24;
        ELSIF x < CONV_SIGNED(CONV_INTEGER(MIN24), ACC_WIDTH) THEN
            RETURN MIN24;
        ELSE
            RETURN x(DATA_WIDTH - 1 DOWNTO 0);
        END IF;
    END FUNCTION;

    SIGNAL y1_reg, y2_reg : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');
    SIGNAL u1_reg, u2_reg : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');
    SIGNAL y_reg          : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');
    SIGNAL done_reg       : STD_LOGIC := '0';

BEGIN

    y_out      <= y_reg;
    y_old1     <= y1_reg;
    y_old2     <= y2_reg;
    u_old1     <= u1_reg;
    u_old2     <= u2_reg;
```

```
BEGIN

    y_out      <= y_reg;
    y_old1     <= y1_reg;
    y_old2     <= y2_reg;
    u_old1     <= u1_reg;
    u_old2     <= u2_reg;
    plant_done <= done_reg;

    PROCESS (clk)
        VARIABLE acc : SIGNED(ACC_WIDTH - 1 DOWNTO 0);
    BEGIN
        IF rising_edge(clk) THEN
            done_reg <= '0';

            IF rst = '1' THEN
                y_reg  <= (OTHERS => '0');
                y1_reg <= (OTHERS => '0');
                y2_reg <= (OTHERS => '0');
                u1_reg <= (OTHERS => '0');
                u2_reg <= (OTHERS => '0');
            ELSIF sample_en = '1' THEN
                acc := PLANT_P1 * y1_reg;
                acc := acc + PLANT_P2 * y2_reg;
                acc := acc + PLANT_B0 * u_now;
                acc := acc + PLANT_B1 * u1_reg;
                acc := acc + PLANT_B2 * u2_reg;
                acc := arshift14(acc);

                y_reg  <= sat24(acc);
                y2_reg <= y1_reg;
                y1_reg <= sat24(acc);
                u2_reg <= u1_reg;
                u1_reg <= u_now;
                done_reg <= '1';
            END IF;
        END IF;
    END PROCESS;
```


PFIEV

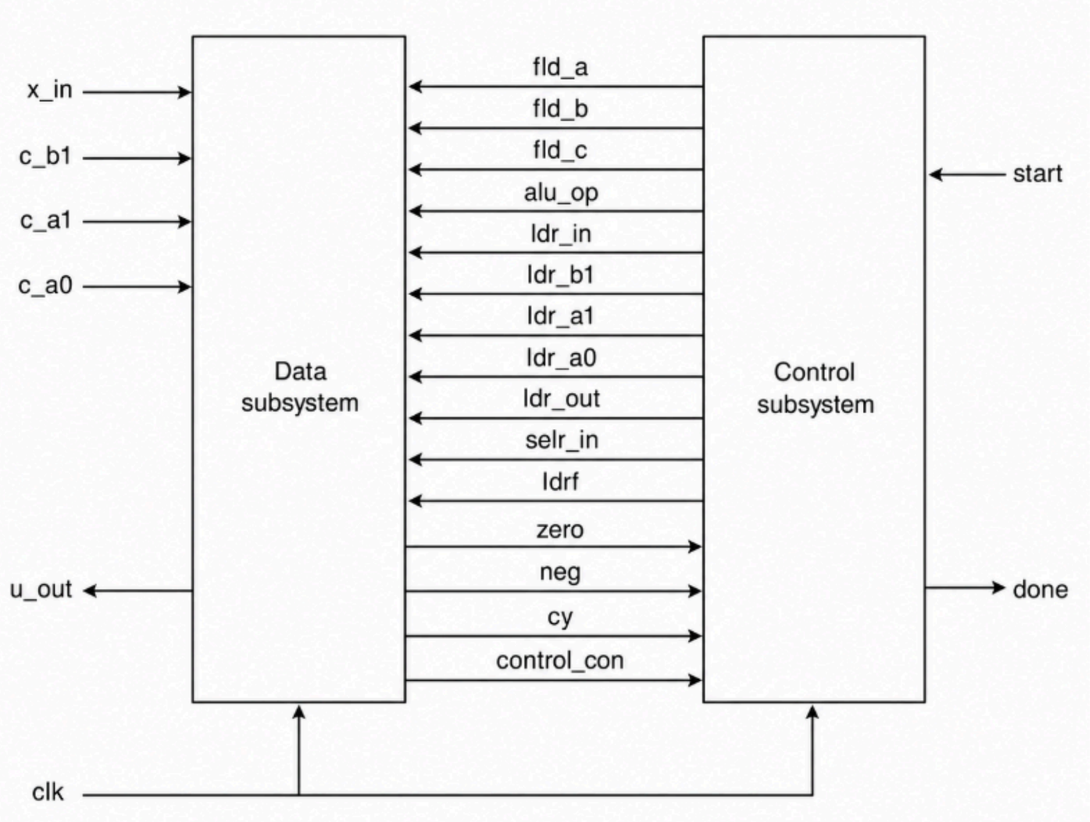
THIẾT KẾ CPU

CÁC THÀNH PHẦN CHÍNH CỦA CPU

- Khối dữ liệu
  - Mô-đun lưu trữ: các thanh ghi lưu e(k), e(k−1), u(k−1)
  - Mô-đun chức năng: ALU thực hiện các phép toán ADD, SUB, MUL
  - Đường truyền dữ liệu: bus dữ liệu, bộ chọn MUX và dây nối
  - Điểm điều khiển: tín hiệu chọn thanh ghi, chọn phép toán ALU và điều khiển ngõ ra
  - Điểm điều kiện: tín hiệu start, done, zero, negative, carry
- Khối điều khiển
  - Định dạng vi lệnh 24-bit
  - Vi chương trình thực hiện phương trình Phase Lag: u(k) = b1u(k−1) + a0e(k) + a1e(k−1)

THIẾT KẾ CPU

Mô tả cấu trúc CPU



- Toàn bộ hệ thống CPU được hình thành từ hai khối chính: Khối dữ liệu và Khối điều khiển.
- Khối dữ liệu thực hiện các phép toán số học 24bit-Q14 cho bộ điều khiển Phase Lag.
- Khối điều khiển nhận các tín hiệu điều kiện từ khối dữ liệu và tạo ra các tín hiệu điều khiển để điều khiển hoạt động của khối dữ liệu.
- Các tín hiệu dữ liệu được biểu diễn theo định dạng signed 24-bit Q14, còn địa chỉ thanh ghi được biểu diễn bằng 4 bit.
- CPU thực hiện phương trình điều khiển Phase Lag:

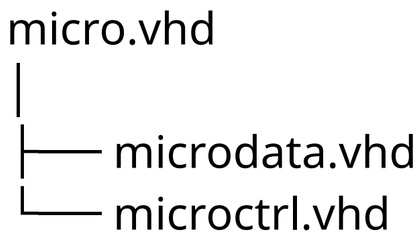
  $u(k) = b1u(k-1) + a0e(k) + a1e(k-1)$
- Mỗi vi lệnh được thực hiện trong một chu kỳ xung clock.

PFIEV

THIẾT KẾ CPU

Mô tả cấu trúc CPU

micro.vhd




```
ENTITY micro IS
    PORT (
        start                                : IN  STD_LOGIC;
        err_in                               : IN  SIGNED(DATA_WIDTH - 1 D
        coeff_bl, coeff_al, coeff_a0         : IN  SIGNED(DATA_WIDTH - 1 D
        u_out                                : OUT SIGNED(DATA_WIDTH - 1 D
        ctrl_done                            : OUT STD_LOGIC;
        clk                                  : IN  STD_LOGIC
    );
END micro;

ARCHITECTURE structural OF micro IS

    SIGNAL fld_a, fld_b, fld_c   : UNSIGNED(3 DOWNTO 0);
    SIGNAL alu_op                : UNSIGNED(1 DOWNTO 0);
    SIGNAL zero, neg, cy         : STD_LOGIC;
    SIGNAL ldr_in, ldr_out, ldrf : STD_LOGIC;
    SIGNAL selr_in               : UNSIGNED(2 DOWNTO 0);
    SIGNAL ldr_bl, ldr_al, ldr_a0 : STD_LOGIC;
    SIGNAL control_con           : STD_LOGIC;
```

```
BEGIN

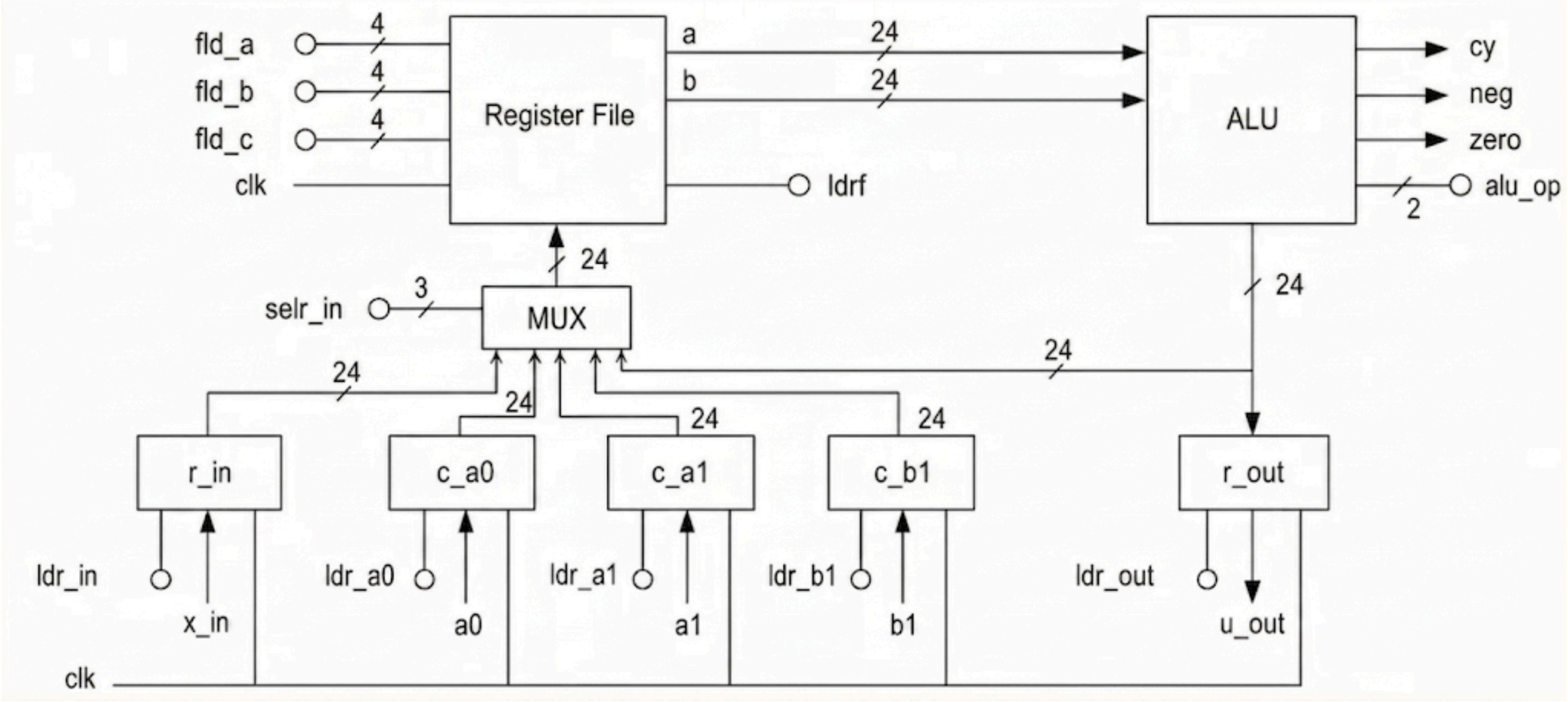
    u1: microdata
        PORT MAP (
            err_in      => err_in,
            coeff_bl    => coeff_bl,
            coeff_al    => coeff_al,
            coeff_a0    => coeff_a0,
            fld_a       => fld_a,
            fld_b       => fld_b,
            fld_c       => fld_c,
            alu_op      => alu_op,
            ldr_in      => ldr_in,
            ldr_bl      => ldr_bl,
            ldr_al      => ldr_al,
            ldr_a0      => ldr_a0,
            ldr_out     => ldr_out,
            selr_in     => selr_in,
            ldrf        => ldrf,
            zero        => zero,
            neg         => neg,
            cy          => cy,
            u_out       => u_out,
            clk         => clk,
            control_con => control_con
        );
```

```
    u2: microctrl
        GENERIC MAP (cssize => CSSIZE)
        PORT MAP (
            start       => start,
            zero        => zero,
            neg         => neg,
            cy          => cy,
            fld_a       => fld_a,
            fld_b       => fld_b,
            fld_c       => fld_c,
            alu_op      => alu_op,
            ldr_in      => ldr_in,
            ldr_bl      => ldr_bl,
            ldr_al      => ldr_al,
            ldr_a0      => ldr_a0,
            ldr_out     => ldr_out,
            selr_in     => selr_in,
            ldrf        => ldrf,
            ctrl_done   => ctrl_done,
            clk         => clk,
            control_con => control_con
        );

END structural;
```

KHỐI DỮ LIỆU

Data Subsystem (microdata.vhd) là datapath của hệ thống, chịu trách nhiệm thực hiện tất cả các phép tính số học. Nó bao gồm:



KHỐI DỮ LIỆU

control signal	meaning
alu_op	operation done in ALU
ldr_a0	load a0
ldr_a1	load a1
ldr_b1	load b1
ldr_in	load r_in
ldr_out	load r_out
fld_a	address for read port a
fld_b	address for read port b
fld_c	address for write port c
selr_in	select data to write
ldrf	load register file (write)

zero	alu_out=0	result is zero
neg	alu_out<0	result is negative
cy	carry	result generated carry

ALU (Arithmetic Logic Unit)

alu_op	Phép toán	Mô tả
"00"	C = A + B	Cộng có dấu 24-bit
"01"	C = A - B	Trừ có dấu 24-bit
"10"	C = A x B	Nhân có dấu, làm tròn rồi lấy bits
"11"	C = A + 1	Tăng 1

KHỐI DỮ LIỆU

control signal	meaning
alu_op	operation done in ALU
ldr_a0	load a0
ldr_a1	load a1
ldr_b1	load b1
ldr_in	load r_in
ldr_out	load r_out
fld_a	address for read port a
fld_b	address for read port b
fld_c	address for write port c
selr_in	select data to write
ldrf	load register file (write)

zero	alu_out=0	result is zero
neg	alu_out<0	result is negative
cy	carry	result generated carry

MUX selr_in

selr_in	Dữ liệu được ghi vào register file
“000"	alu_out
“001"	r_in, tức e(k)
“010"	c_b1 → hệ số b1
“011"	c_a1 → hệ số a1
“100"	c_a0 → hệ số a0

KHỐI DỮ LIỆU

microdata.vhd

Phân bố thanh ghi (Register Allocation)

Thanh ghi	Tên	Mục đích	Tính chất
R0	ZERO	Hằng số 0	Không đổi
R1	e(k)	Sai số hiện tại	Cập nhật mỗi chu kỳ
R2	ACC	Bộ tích lũy / kết quả	Trung gian
R3	TMP1	Tạm thời (hệ số b1, sau đó là b1×u_old)	Trung gian
R4	TMP2	Tạm thời (hệ số a1, sau đó là a1×e_old)	Trung gian
R5	—	Chưa sử dụng	Dự phòng
R6	e(k-1)	Sai số chu kỳ trước	Duy trì giữa các lần
R7	u(k-1)	Tín hiệu điều khiển chu kỳ trước	Duy trì giữa các lần

```
ENTITY microdata IS
    PORT (
        err_in, coeff_b1, coeff_a1, coeff_a0 : IN  SIGNED(DATA_WIDTH - 1 DOWNTO 0);
        fld_a, fld_b, fld_c                  : IN  UNSIGNED(3 DOWNTO 0);
        alu_op                               : IN  UNSIGNED(1 DOWNTO 0);
        ldr_in, ldr_b1, ldr_a1, ldr_a0, ldr_out : IN  STD_LOGIC;
        selr_in                              : IN  UNSIGNED(2 DOWNTO 0);
        ldrf                                 : IN  STD_LOGIC;
        zero, neg, cy                        : OUT STD_LOGIC;
        u_out                                : OUT SIGNED(DATA_WIDTH - 1 DOWNTO 0);
        clk                                  : IN  STD_LOGIC;
        control_con                          : IN  STD_LOGIC
    );
END microdata;
```


PFIEV

THIẾT KẾ CPU

```
ARCHITECTURE behavioral OF microdata IS

    FUNCTION sat24(x : SIGNED(ACC_WIDTH - 1 DOWNTO 0)) RETURN SIGNED IS
        CONSTANT MAX24 : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := CONV_SIGNED(8388607, DATA_WIDTH);
        CONSTANT MIN24 : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := CONV_SIGNED(-8388608, DATA_WIDTH);
    BEGIN
        IF x > CONV_SIGNED(CONV_INTEGER(MAX24), ACC_WIDTH) THEN
            RETURN MAX24;
        ELSIF x < CONV_SIGNED(CONV_INTEGER(MIN24), ACC_WIDTH) THEN
            RETURN MIN24;
        ELSE
            RETURN x(DATA_WIDTH - 1 DOWNTO 0);
        END IF;
    END FUNCTION;

    TYPE reg_fileT IS ARRAY (0 TO 7) OF SIGNED(DATA_WIDTH - 1 DOWNTO 0);
    SIGNAL RF : reg_fileT := (OTHERS => (OTHERS => '0'));

    SIGNAL r_in  : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');
    SIGNAL c_b1  : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');
    SIGNAL c_a0  : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');
    SIGNAL c_a1  : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');

    FUNCTION rf_index(sel : UNSIGNED(3 DOWNTO 0)) RETURN NATURAL IS
    BEGIN
        IF sel(3) = '1' THEN
            RETURN 0;
        END IF;
        RETURN CONV_INTEGER(sel(2 DOWNTO 0));
    END FUNCTION;
```

```
BEGIN

    PROCESS (clk)
        VARIABLE A, B, C      : SIGNED(DATA_WIDTH - 1 DOWNTO 0);
        VARIABLE alu_out      : SIGNED(DATA_WIDTH - 1 DOWNTO 0);
        VARIABLE alu_out_reg  : SIGNED(DATA_WIDTH DOWNTO 0);
        VARIABLE alu_out_mul  : SIGNED(ACC_WIDTH - 1 DOWNTO 0);
        VARIABLE zzero, nneg, ccy : STD_LOGIC;
        CONSTANT Z : SIGNED(DATA_WIDTH - 1 DOWNTO 0) := (OTHERS => '0');

        PROCEDURE alu (
            zzero, nneg, ccy : OUT STD_LOGIC;
            result           : OUT SIGNED(DATA_WIDTH - 1 DOWNTO 0);
            a, b             : IN  SIGNED(DATA_WIDTH - 1 DOWNTO 0);
            op               : IN  UNSIGNED(1 DOWNTO 0)
        ) IS
        BEGIN
            CASE op IS
                WHEN "00" =>
                    alu_out_reg := (a(DATA_WIDTH - 1) & a) + (b(DATA_WIDTH - 1) & b);
                    result := sat24(CONV_SIGNED(CONV_INTEGER(alu_out_reg), ACC_WIDTH));
                    ccy := alu_out_reg(DATA_WIDTH);
                    zzero := '0';
                    IF result = Z THEN zzero := '1'; END IF;
                    nneg := result(DATA_WIDTH - 1);

                WHEN "01" =>
                    alu_out_reg := (a(DATA_WIDTH - 1) & a) - (b(DATA_WIDTH - 1) & b);
                    result := sat24(CONV_SIGNED(CONV_INTEGER(alu_out_reg), ACC_WIDTH));
                    ccy := '0';
                    zzero := '0';
                    IF result = Z THEN zzero := '1'; END IF;
                    nneg := result(DATA_WIDTH - 1);

                WHEN "10" =>
                    alu_out_mul := a * b;
                    alu_out_mul := arshift14(alu_out_mul);
                    result := sat24(alu_out_mul);
                    ccy := '0';
```


PFIEV

THIẾT KẾ CPU

```
                zzero := '0';
                IF result = Z THEN zzero := '1'; END IF;
                nneg := result(DATA_WIDTH - 1);

            WHEN "11" =>
                alu_out_reg := (a(DATA_WIDTH - 1) & a) + CONV_SIGNED(1, DATA_WIDTH + 1);
                result := sat24(CONV_SIGNED(CONV_INTEGER(alu_out_reg), ACC_WIDTH));
                ccy := '0';
                zzero := '0';
                IF result = Z THEN zzero := '1'; END IF;
                nneg := result(DATA_WIDTH - 1);

            WHEN OTHERS =>
                result := (OTHERS => '0');
                zzero := '0';
                nneg := '0';
                ccy := '0';
        END CASE;
    END PROCEDURE;

BEGIN
    A := RF(rf_index(fld_a));
    B := RF(rf_index(fld_b));

    IF control_con = '0' THEN
        alu(zzero, nneg, ccy, alu_out, A, B, alu_op);
        zero <= zzero;
        neg  <= nneg;
        cy   <= ccy;
    END IF;

    CASE selr_in IS
        WHEN "000" => C := alu_out;
        WHEN "001" => C := r_in;
        WHEN "010" => C := c_bl;
        WHEN "011" => C := c_al;
        WHEN "100" => C := c_a0;
        WHEN OTHERS => C := (OTHERS => '0');
    END CASE;
```

```
        IF rising_edge(clk) THEN
            IF ldr_in = '1' THEN r_in <= err_in; END IF;
            IF ldr_bl = '1' THEN c_bl <= coeff_bl; END IF;
            IF ldr_al = '1' THEN c_al <= coeff_al; END IF;
            IF ldr_a0 = '1' THEN c_a0 <= coeff_a0; END IF;
            IF ldr_out = '1' THEN u_out <= alu_out; END IF;
            IF ldrf = '1' THEN RF(rf_index(fld_c)) <= C; END IF;
        END IF;
    END PROCESS;

END behavioral;
```


PFIEV

KHỐI ĐIỀU KHIỂN

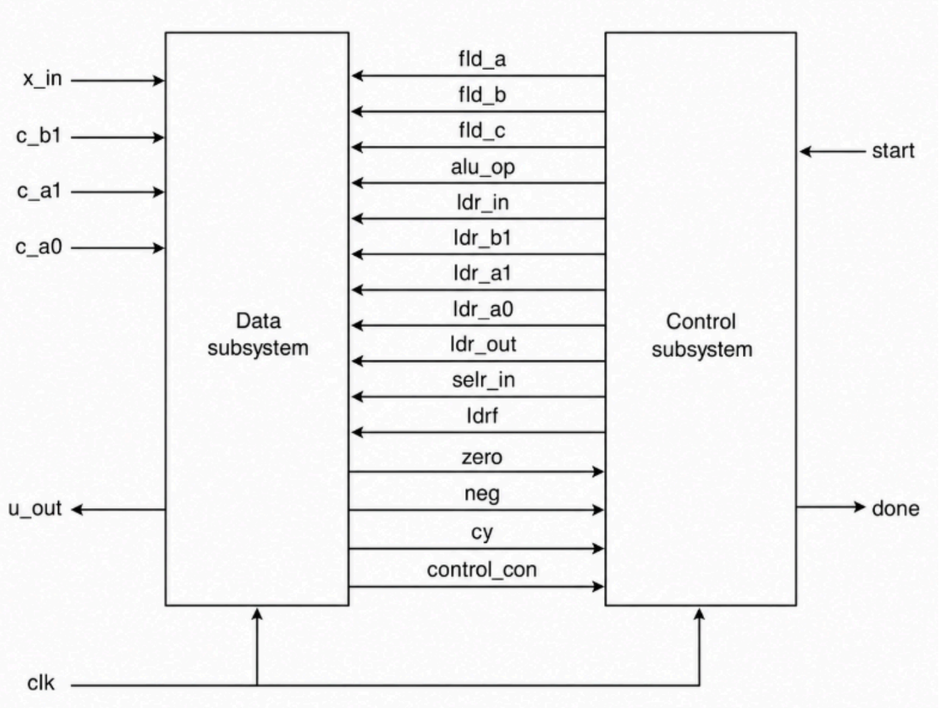
Cấu trúc vi lệnh (26-bit Microinstruction)

Operation

Mode=0 [25]	alu_op [24:23]	fld_a [22:19]	fld_b [18:15]	fld_c [14:11]	ldrf [10]	ldr_in [9]	ldr_b1 [8]	ldr_a1 [7]	ldr_a0 [6]	ldr_out [5]	selr_in [4:2]	D1 [1]	D0 [0]

Branch

Mode=1 [25]	condition [24:23]	cond_val [22]	branch_address [21:0]



Điều kiện rẽ nhánh

condition[1:0]	Tín hiệu kiểm tra	Ý nghĩa
"00"	start	Bắt đầu tính toán
"01"	zero	Kết quả ALU = 0
"10"	neg	Kết quả ALU < 0
"11"	cy	Có carry từ ALU

Inputs: start, zero, neg, cy

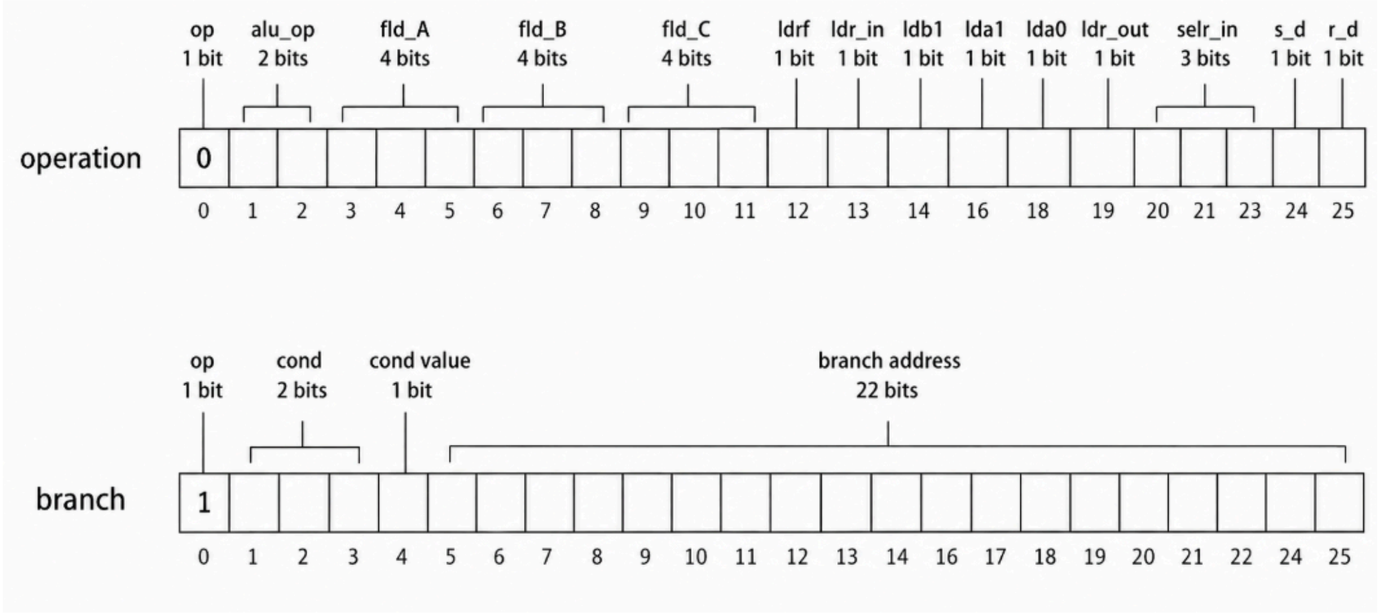
Outputs: fld_a, fld_b, fld_c, alu_op, ldr_in, ldr_a0, ldr_a1, ldr_b1, ldr_out, ldrf, selr_in, D1, D0

PFIEV

KHỐI ĐIỀU KHIỂN

- Ngõ vào: start, zero, neg, cy
- Ngõ ra: fld_A, fld_B, fld_C, alu_op, ldr_in, ldb1, lda1, lda0, ldr_out, selr_in, ldrf, done
- Tín hiệu done được đặt hoặc xóa bằng cách đưa giá trị 1 vào các trường s_d hoặc r_d
- Vi lệnh gồm hai dạng: vi lệnh thao tác và vi lệnh rẽ nhánh

Condition	Code
start	00
zero	01
neg	10
cy	11



KHỐI ĐIỀU KHIỂN microctrl.vhd

```
ENTITY microctrl IS
    GENERIC (
        cssize : NATURAL := 17   -- Number of microinstructions
    );
    PORT (
        -- Input status flags
        start     : IN  STD_LOGIC;   -- Start new computation
        zero      : IN  STD_LOGIC;   -- ALU zero flag
        neg       : IN  STD_LOGIC;   -- ALU negative flag
        cy        : IN  STD_LOGIC;   -- ALU carry flag

        -- Output control signals to datapath
        fld_a     : OUT UNSIGNED(3 DOWNTO 0);  -- Source register A
        fld_b     : OUT UNSIGNED(3 DOWNTO 0);  -- Source register B
        fld_c     : OUT UNSIGNED(3 DOWNTO 0);  -- Destination register
        alu_op    : OUT UNSIGNED(1 DOWNTO 0);  -- ALU operation
        ldr_in    : OUT STD_LOGIC;   -- Load error input
        ldr_b1    : OUT STD_LOGIC;   -- Load B1 coefficient
        ldr_a1    : OUT STD_LOGIC;   -- Load A1 coefficient
        ldr_a0    : OUT STD_LOGIC;   -- Load A0 coefficient
        ldr_out   : OUT STD_LOGIC;   -- Load output register
        selr_in   : OUT UNSIGNED(2 DOWNTO 0);  -- Write-back source select
        ldrf      : OUT STD_LOGIC;   -- Register file write enable
        ctrl_done : OUT STD_LOGIC := '0';   -- Computation complete

        -- System clock
        clk       : IN  STD_LOGIC;

        -- Control connection
        control_con : OUT STD_LOGIC
    );
END microctrl;
```

```
ARCHITECTURE behav_microprogr OF microctrl IS

    SIGNAL csar   : NATURAL RANGE 0 TO cssize-1 := 0;
    SIGNAL uinstr : UNSIGNED(25 DOWNTO 0) := (OTHERS => '0');
    ALIAS mode      : STD_LOGIC              IS uinstr(25);
    ALIAS condition : UNSIGNED(1 DOWNTO 0)   IS uinstr(24 DOWNTO 23);
    ALIAS cond_val  : STD_LOGIC              IS uinstr(22);

BEGIN

    PROCESS (clk)
        VARIABLE index : UNSIGNED(21 DOWNTO 0);
    BEGIN
        IF (clk'EVENT AND clk = '1') THEN

            IF (csar = cssize-1) THEN
                -- Last ROM entry is a safety return to the wait-start state.
                -- This prevents fall-through to address 17 when a status flag
                -- has an unexpected value at the end of a transaction.
                csar <= 1;

            ELSIF (mode = '0') THEN
                -- Operational mode: simply advance to next instruction
                csar <= csar + 1;
            ELSE
                -- Branch mode: evaluate condition
                CASE condition IS

                    WHEN "00" =>
                        IF (start = cond_val) THEN
                            index := uinstr(21 DOWNTO 0);
                            csar  <= CONV_INTEGER(index);
                        ELSE
                            csar <= csar + 1;
                        END IF;
```


PFIEV

THIẾT KẾ CPU

KHỐI ĐIỀU KHIỂN

```
                WHEN "01" =>
                    IF (zero = cond_val) THEN
                        index := uinstr(21 DOWNTO 0);
                        csar  <= CONV_INTEGER(index);
                    ELSE
                        csar <= csar + 1;
                    END IF;

                WHEN "10" =>
                    IF (neg = cond_val) THEN
                        index := uinstr(21 DOWNTO 0);
                        csar  <= CONV_INTEGER(index);
                    ELSE
                        csar <= csar + 1;
                    END IF;

                WHEN "11" =>
                    IF (cy = cond_val) THEN
                        index := uinstr(21 DOWNTO 0);
                        csar  <= CONV_INTEGER(index);
                    ELSE
                        csar <= csar + 1;
                    END IF;

                WHEN OTHERS => NULL;
            END CASE;
        END IF;

    END IF;
END PROCESS;
```

```
PROCESS (csar)
    TYPE csarray IS ARRAY (0 TO cssize-1) OF UNSIGNED(25 DOWNTO 0);
    VARIABLE cs : csarray := (

        0 => "00000000000000010000000001",
        1 => "10000000000000000000000001",
        2 => "00000000000001111111000001",
        3 => "00000000000001100000000101",
        4 => "00000000000010100000010001",
        5 => "01000100001001010000000001",
        6 => "00000000000010010000001101",
        7 => "01001000110010010000000001",
        8 => "00000100100001010000000001",
        9 => "00000000000001110000001001",
        10 => "01000110111001110000000001",
        11 => "00000100011001010000000001",
        12 => "00000010000011010000000001",
        13 => "00000100000011110000000001",
        14 => "00000100000000000000100010",
        15 => "11100000000000000000000001",
        16 => "11100000000000000000000001"
    );
BEGIN
    uinstr <= cs(csar);
    control_con <= uinstr(25);
    CASE uinstr(25) IS
        WHEN '0' =>
            alu_op  <= uinstr(24 DOWNTO 23);

            fld_a   <= uinstr(22 DOWNTO 19);
            fld_b   <= uinstr(18 DOWNTO 15);
            fld_c   <= uinstr(14 DOWNTO 11);

            ldrf    <= uinstr(10);

            ldr_in  <= uinstr(9);
            ldr_bl  <= uinstr(8);
            ldr_al  <= uinstr(7);
            ldr_a0  <= uinstr(6);
                ldr_out <= uinstr(5);

                selr_in <= uinstr(4 DOWNTO 2);
            WHEN '1' =>
                ldrf    <= '0';
                ldr_out <= '0';
                ldr_in  <= '0';
                ldr_bl  <= '0';
                ldr_al  <= '0';
                ldr_a0  <= '0';

            WHEN OTHERS => NULL;
        END CASE;

END PROCESS;

PROCESS (clk)
BEGIN
    IF rising_edge(clk) THEN
        ctrl_done <= '0';
        IF csar = 14 AND uinstr(25) = '0' THEN
            ctrl_done <= '1';
        END IF;
    END IF;
END PROCESS;

END behav_microprogr;
```


PFIEV

VI CHƯƠNG TRÌNH

- Mục tiêu:

$$u(k) = b_1 u(k-1) + a_0 e(k) + a_1 e(k-1)$$

- CSAR (Control Store Address Register) là con trỏ chương trình, trỏ đến vi lệnh hiện tại. Cơ chế hoạt động:
  - Nếu mode = '0': CSAR ← CSAR + 1
  - Nếu mode = '1' và điều kiện đúng
    → CSAR ← branch_address (rẽ nhánh)
  - Nếu mode = '1' và điều kiện sai
    → CSAR ← CSAR + 1 (bỏ qua nhánh)

- Biểu diễn nhị phân của vi chương trình

```
PROCESS (csar)
    TYPE csarray IS ARRAY (0 TO cssize-1) OF UNSIGNED(25 DOWNTO 0);
    VARIABLE cs : csarray := (

        0 => "00000000000000010000000001",
        1 => "10000000000000000000000001",
        2 => "00000000000000111111000001",
        3 => "00000000000000110000000101",
        4 => "00000000000001010000010001",
        5 => "01000100001001010000000001",
        6 => "00000000000010010000001101",
        7 => "01001000110010010000000001",
        8 => "00000100100001010000000001",
        9 => "00000000000001110000001001",
       10 => "01000110111001110000000001",
       11 => "00000100011001010000000001",
       12 => "00000010000011010000000001",
       13 => "00000100000011110000000001",
       14 => "00000100000000000000100010",
       15 => "11100000000000000000000001",
       16 => "11100000000000000000000001"
    );
```

Mode=1 [25]	condition [24:23]	cond_val [22]	branch_address [21:0]

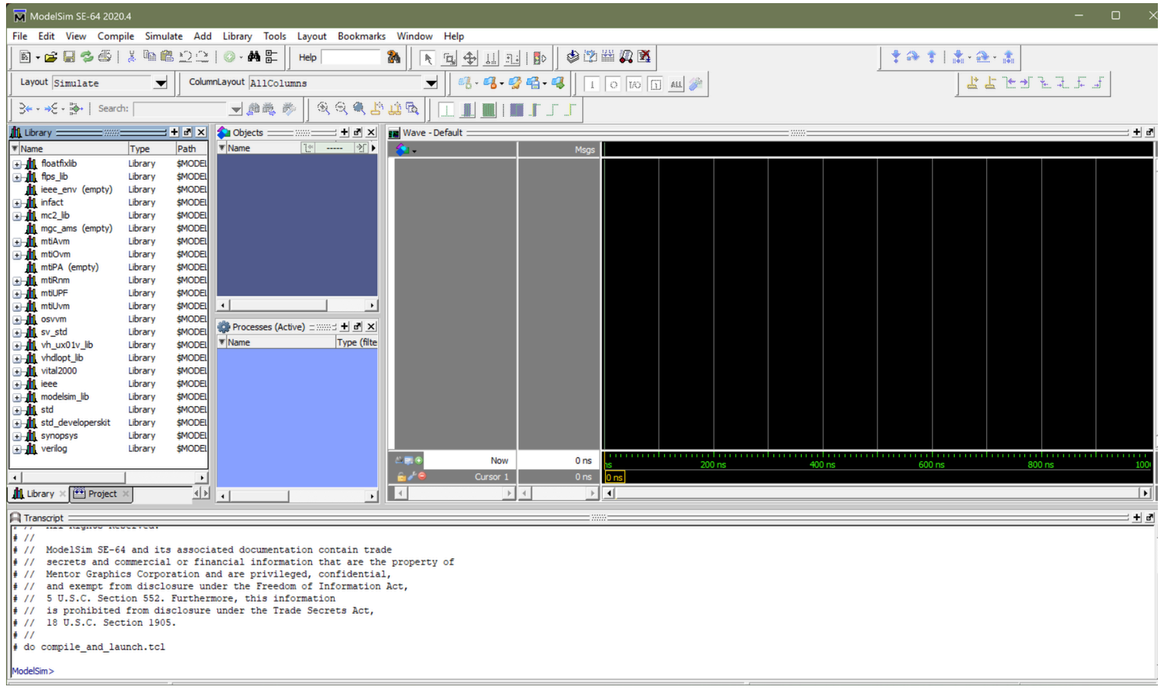
Mode=0 [25]	alu_op [24:23]	fld_a [22:19]	fld_b [18:15]	fld_c [14:11]	ldrf [10]	ldr_in [9]	ldr_b1 [8]	ldr_a1 [7]	ldr_a0 [6]	ldr_out [5]	selr_in [4:2]	D1 [1]	D0 [0]

condition[1:0]	Tín hiệu kiểm tra	Ý nghĩa
"00"	start	Bắt đầu tính toán
"01"	zero	Kết quả ALU = 0
"10"	neg	Kết quả ALU < 0
"11"	cy	Có carry từ ALU

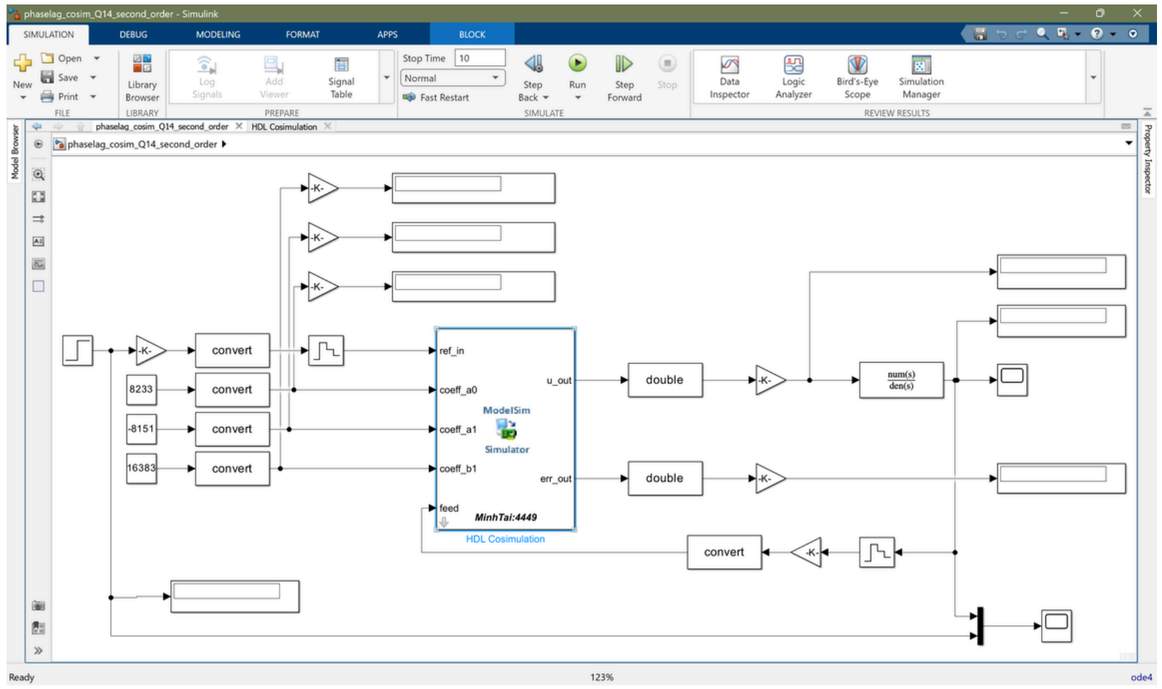
KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Mô phỏng và kiểm tra kết quả bằng ModelSim + Simulink/Matlab

ModelSim



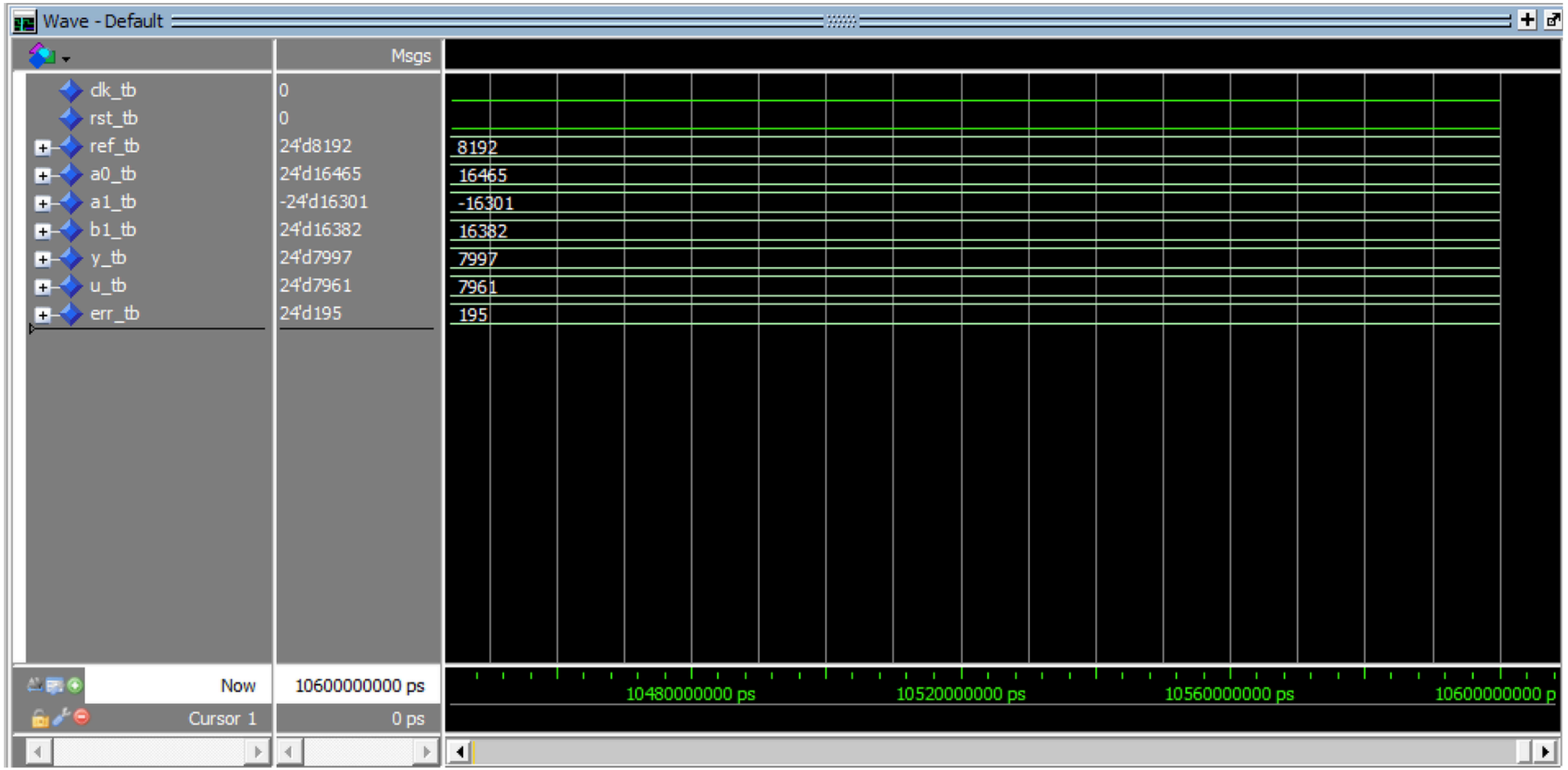
Simulink



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp thiết kế ban đầu: $C(s) = \frac{100(s+1)}{100s+1}$

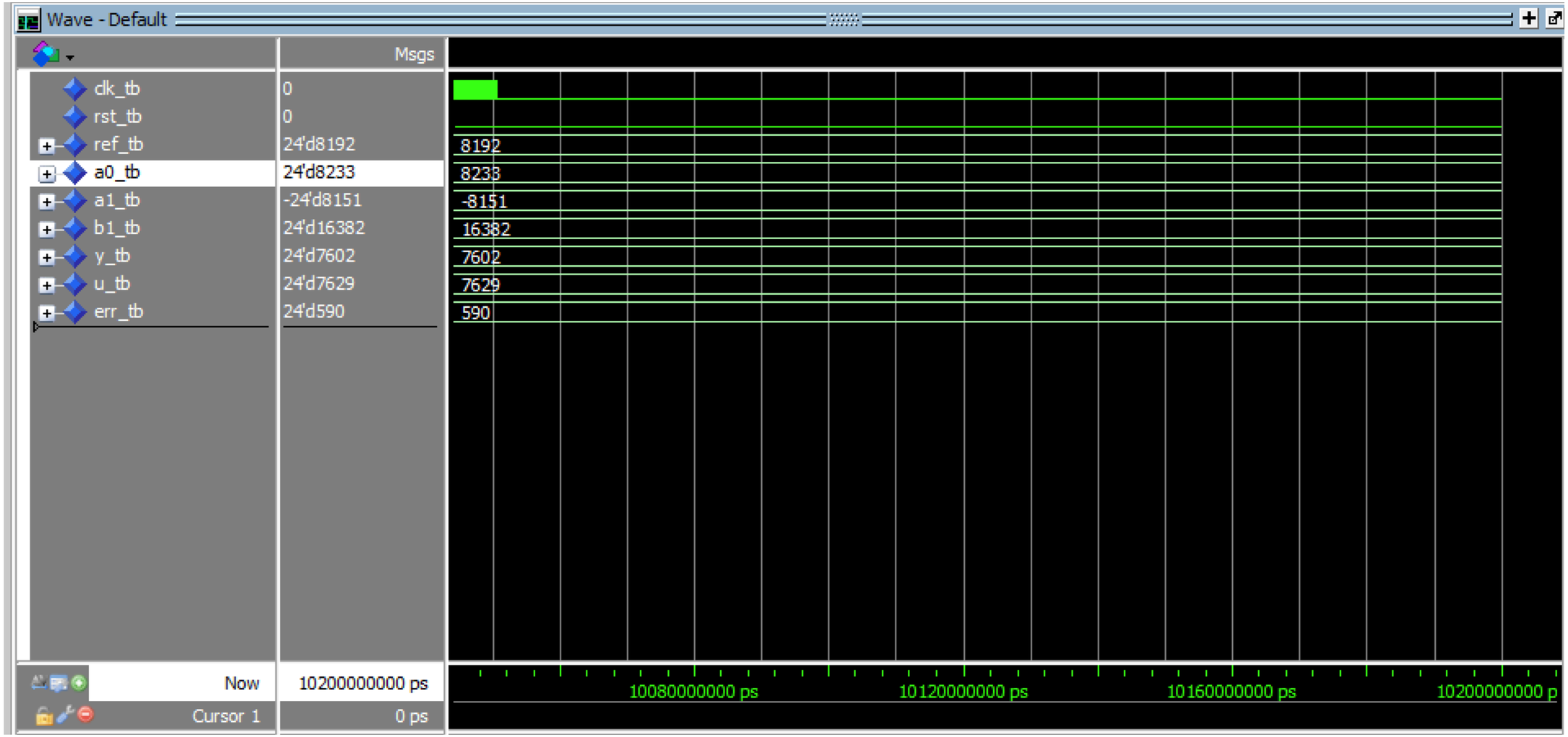
a0 = 1.004950,a1 = -0.994950, b1 = 0.9999 <=> a0 = 16465, a1 = -16301, b1 = 16382



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp: $C(s) = \frac{50(s+1)}{100s+1}$

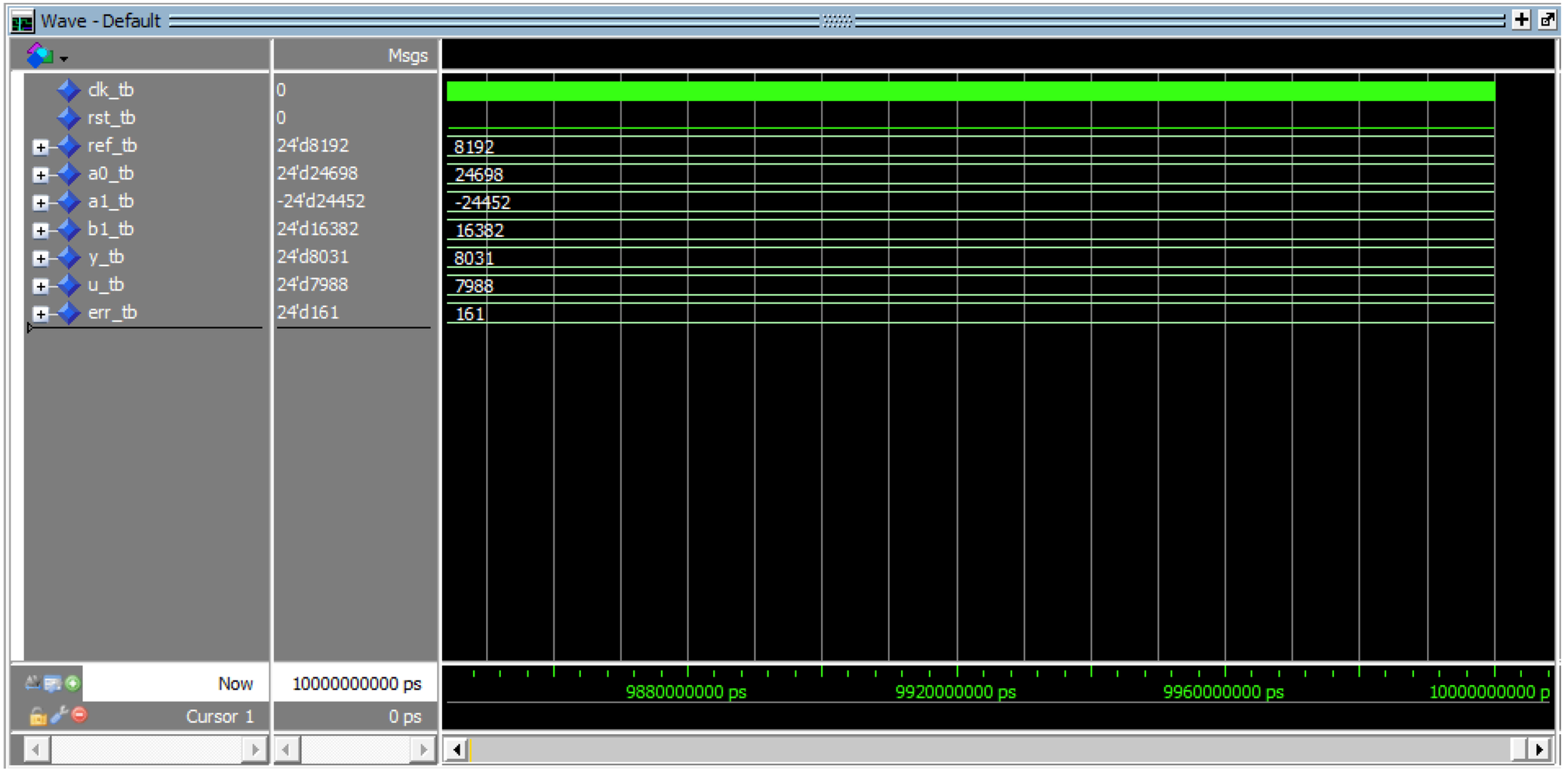
a0 = 0.502487,a1 = -0.497488, b1 = 0.999950 <=> a0 = 8233, a1 = -8151, b1 = 16382



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp: $C(s) = \frac{120(s+1)}{80s+1}$

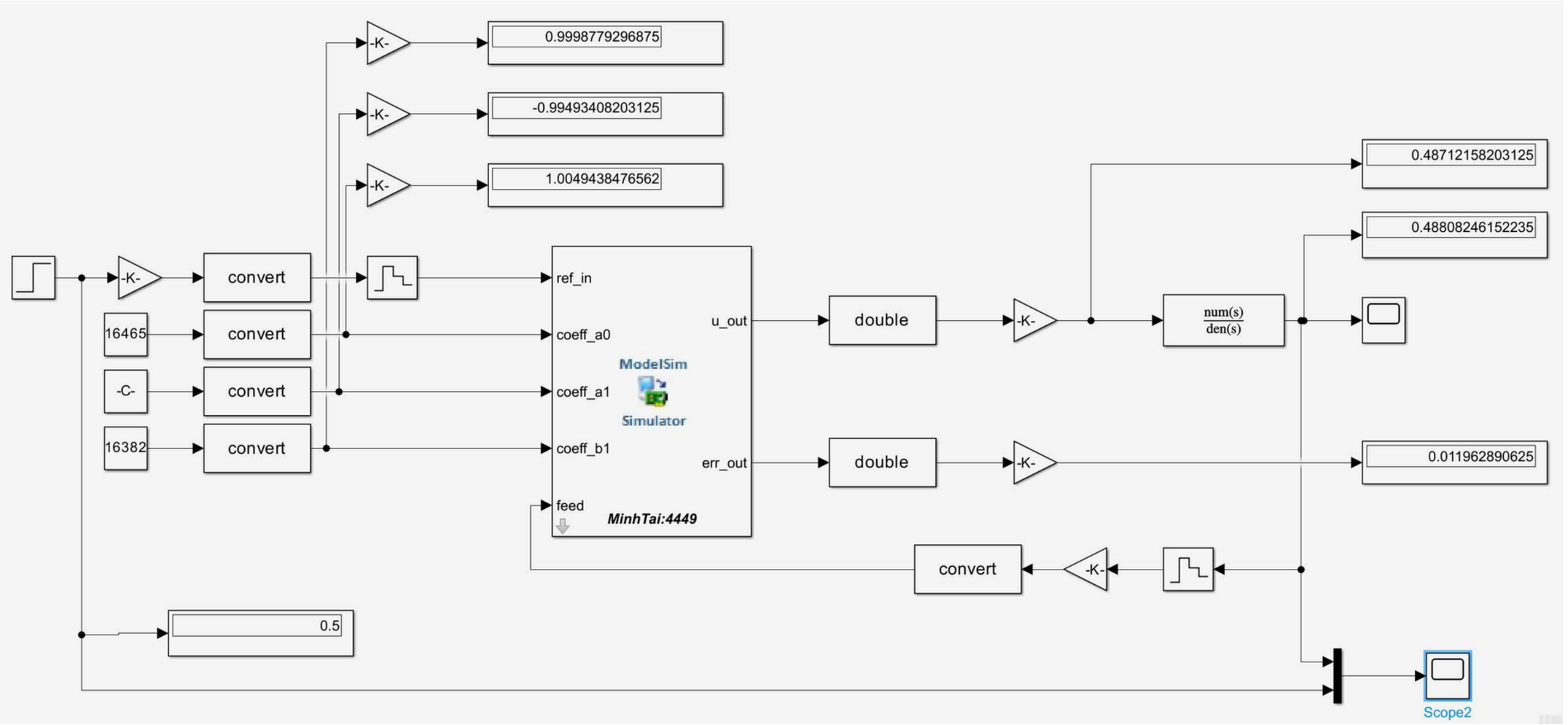
a0 = 1.507406,a1 = -1.492407,b1 = 0.999875 <=> a0 = 24697,a1 = -24452,b1 = 16382



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp thiết kế ban đầu: $C(s) = \frac{100(s+1)}{100s+1}$

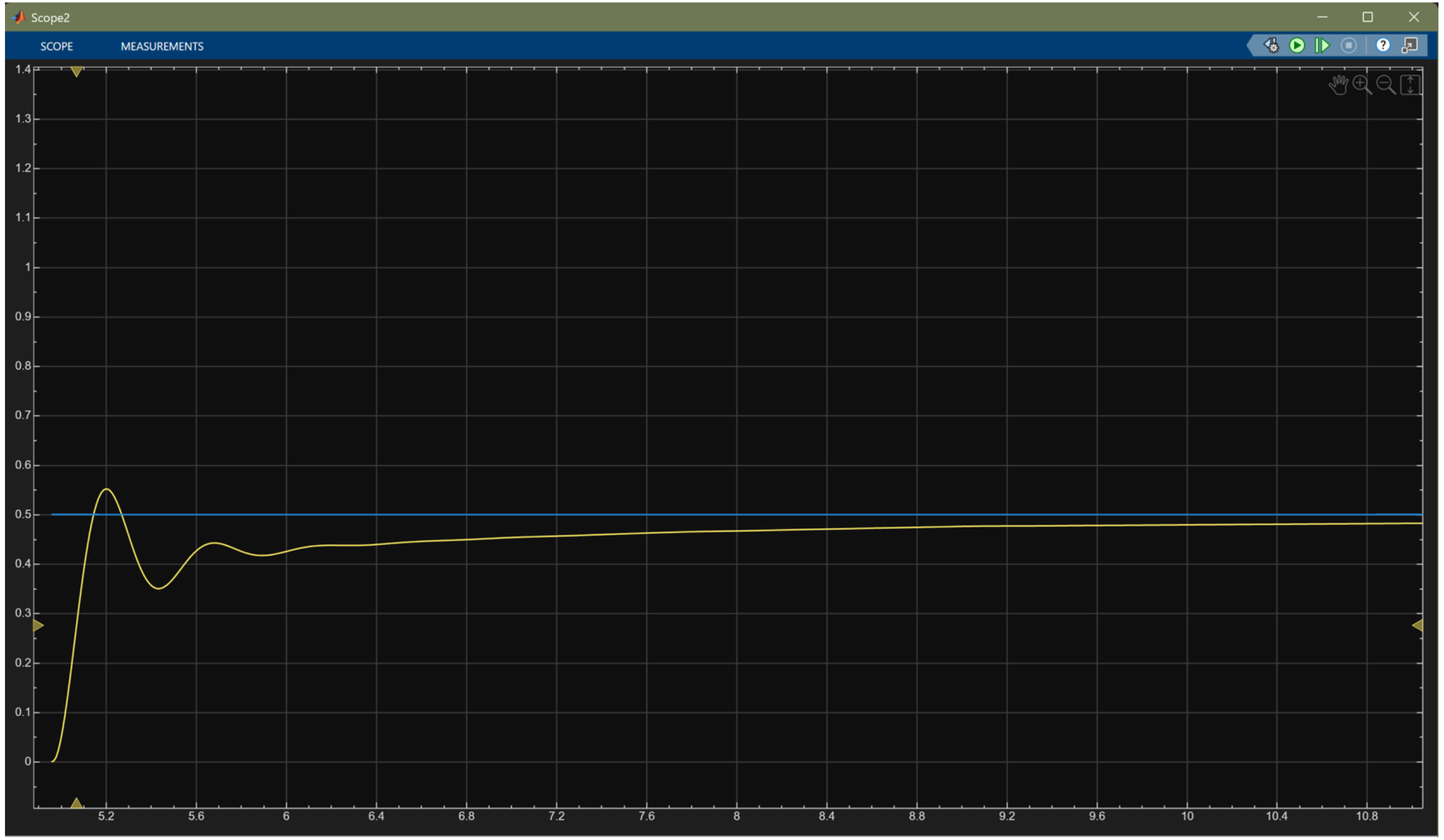
a0 = 1.004950,a1 = -0.994950, b1 = 0.9999 <=> a0 = 16465, a1 = -16301, b1 = 16382



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp thiết kế ban đầu: $C(s) = \frac{100(s+1)}{100s+1}$

a0 = 1.004950,a1 = -0.994950, b1 = 0.9999 <=> a0 = 16465, a1 = -16301, b1 = 16382

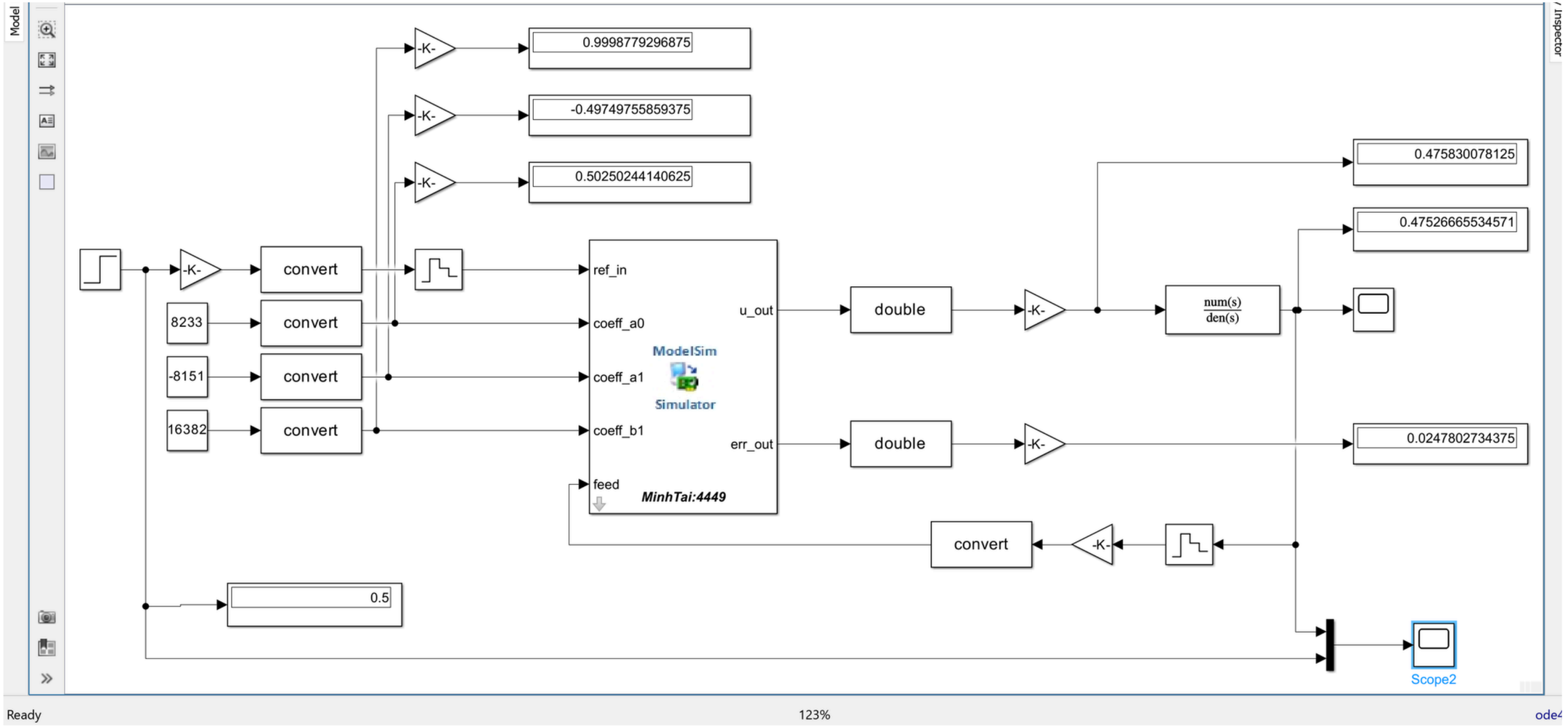


PFIEV

KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp: $C(s) = \frac{50(s+1)}{100s+1}$

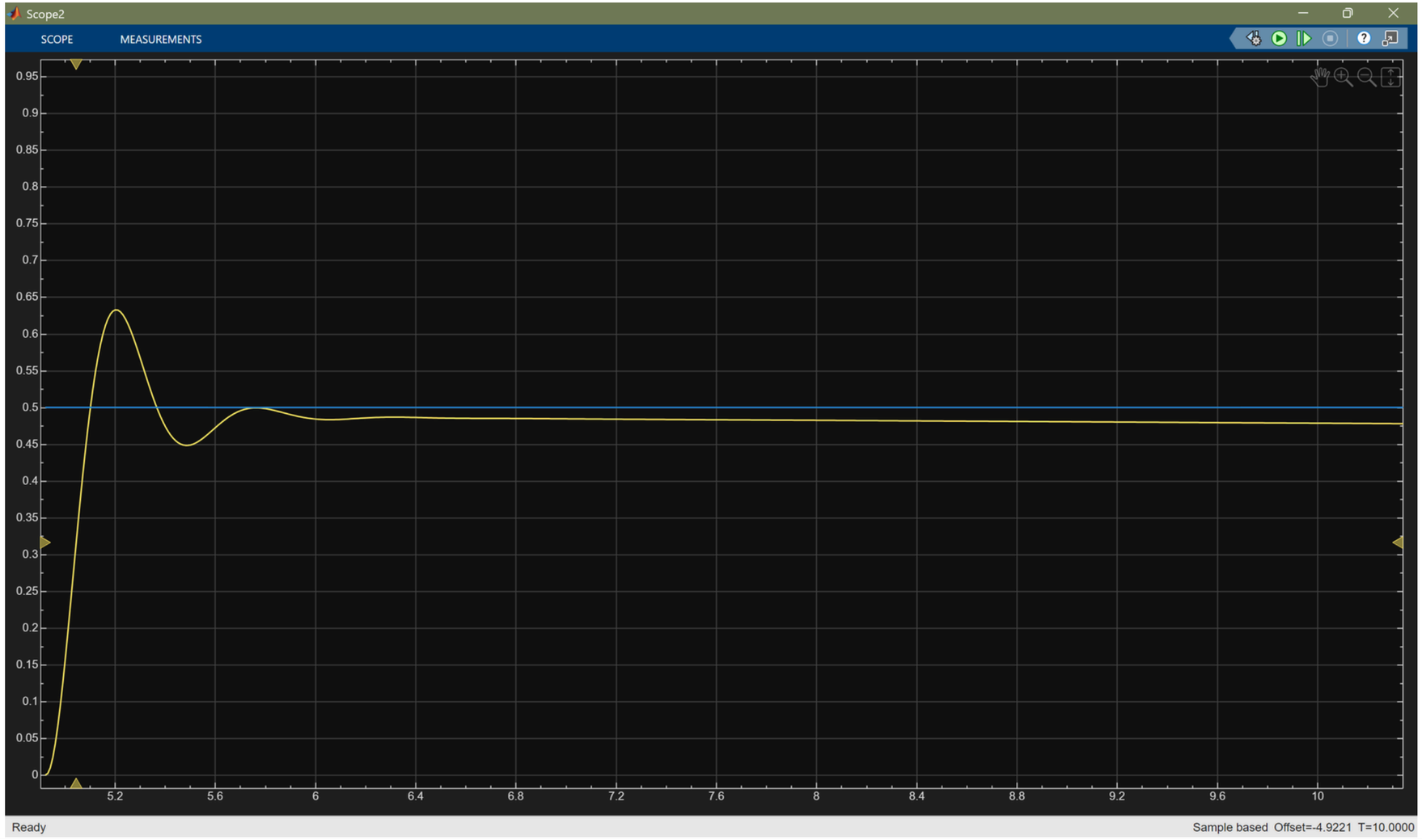
a0 = 0.502487,a1 = -0.497488, b1 = 0.999950 <=> a0 = 8233, a1 = -8151, b1 = 16382



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp: $C(s) = \frac{50(s+1)}{100s+1}$

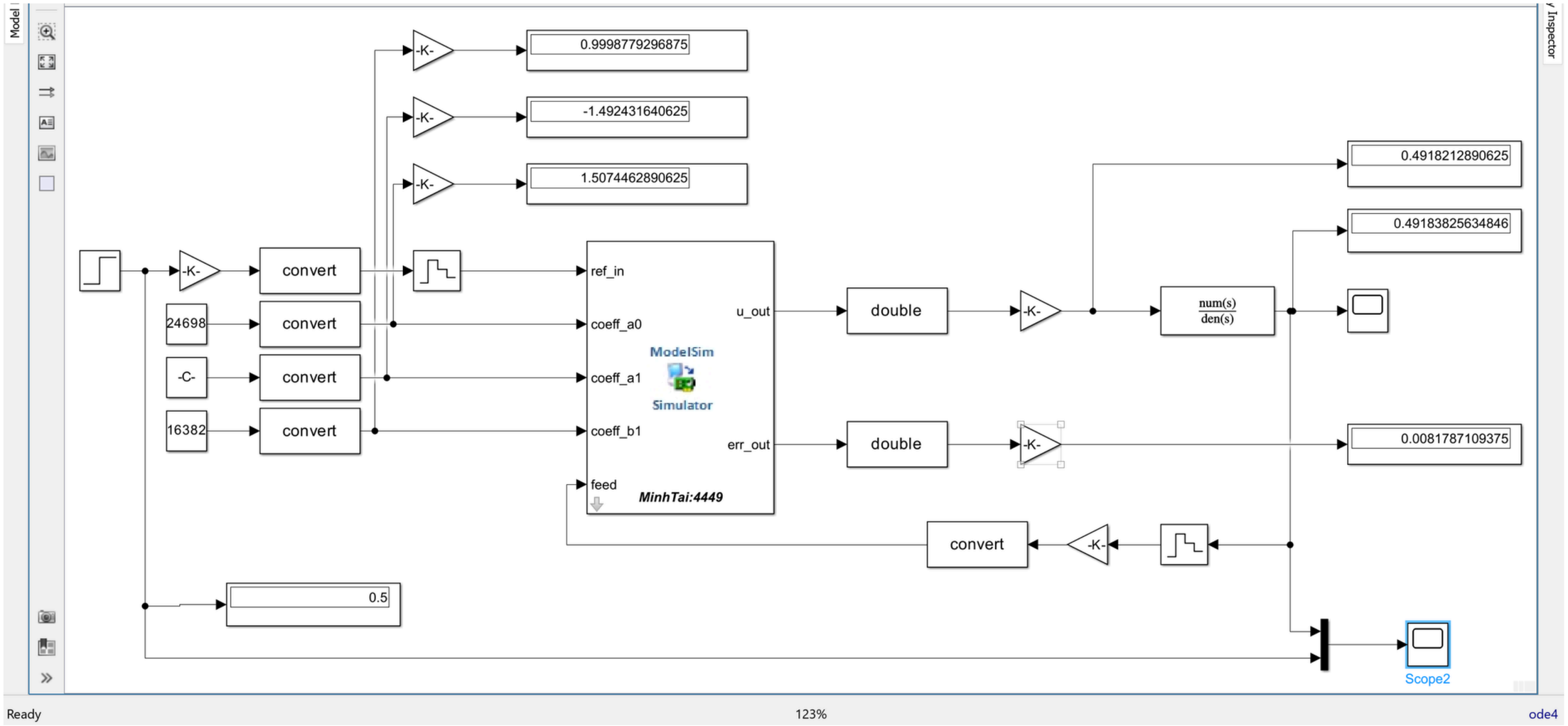
a0 = 0.502487,a1 = -0.497488, b1 = 0.999950 <=> a0 = 8233, a1 = -8151, b1 = 16382



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp: $C(s) = \frac{120(s+1)}{80s+1}$

a0 = 1.507406,a1 = -1.492407,b1 = 0.999875 <=> a0 = 24697,a1 = -24452,b1 = 16382



KẾT QUẢ MÔ PHỎNG VÀ KẾT LUẬN

Trường hợp: $C(s) = \frac{120(s+1)}{80s+1}$

a0 = 1.507406,a1 = -1.492407,b1 = 0.999875 <=> a0 = 24697,a1 = -24452,b1 = 16382

